Cheatsheet — Librerías JavaScript (jQuery y Prototype)

> **Materia:** Programación Orientada a la Web
> **Uso:** repaso rápido y consulta durante el parcial.
> **Base:** teoría de la cátedra (Prototype.js y jQuery 3.x).

Índice

1. Frameworks y librerías JavaScript
2. Prototype JS: idea general
3. Prototype: la función `$()`
4. Prototype: `$F()`, `$$()`, `$A()`, `$H()`, `$R()`
5. Prototype: AJAX (`Ajax.Request`, `Ajax.Updater`)
6. jQuery: qué es y para qué sirve
7. Formas de incluir jQuery
8. Sintaxis: `$(selector).action()`
9. `$(document).ready()`
10. Selectores
11. Eventos
12. Efectos y animaciones
13. Encadenamiento y callbacks
14. Manipulación de HTML: obtener y setear
15. Agregar contenido
16. Eliminar contenido
17. Manipulación de CSS
18. AJAX con jQuery: `load()`
19. AJAX con jQuery: `get()` y `post()`
20. AJAX con jQuery: `ajax()`
21. `noConflict()`
22. Prototype vs. jQuery
23. Errores típicos y checklist
24. Plantilla completa

1. Frameworks y librerías JavaScript

La aparición de JavaScript produjo una gran variedad de **frameworks** o **librerías** para manipular el DOM desde el cliente o trabajar del lado del servidor.

Lado	Frameworks / librerías
Cliente	Prototype.js, jQuery, AngularJS, lightbox (lightbox 2), W3.js
Servidor	Node.js

¿Por qué usar una librería?

- Simplifican tareas repetitivas y verbosas (seleccionar elementos, escuchar eventos, AJAX).
- Resuelven diferencias entre navegadores.
- Reducen la cantidad de código y los errores.
- Costo: una dependencia externa más y una curva de aprendizaje.

2. Prototype JS: idea general

- Se orienta al **desarrollo sencillo y dinámico** de aplicaciones web.
- Es una herramienta que implementa las técnicas **AJAX**.
- Se convirtió en poco tiempo en una **referencia básica de AJAX** y fue la base de muchos otros frameworks; inspiró librerías relacionadas como **jQuery**.
- Su rasgo más conocido es la **función `$()`** (también llamada "función dólar").

3. Prototype: la función `$()`

Es un **atajo** de `document.getElementById()`.

```
// Con JavaScript puro
var elemento = document.getElementById('primero');

// Con Prototype
var elemento = $('primero');
```

JavaScript	Prototype	Qué hace
`document.getElementById('x')`	`$('x')`	Obtiene el elemento con `id="x"`.

> Al principio puede parecer abstracta, pero solo evita escribir `document.getElementById` todo el tiempo.

4. Prototype: `$F()`, `$$()`, `$A()`, `$H()`, `$R()`

Función	Equivalente / uso	Ejemplo
`$('id')`	`document.getElementById()`	`$('primero')`
`$F('id')`	Valor de un elemento de formulario	`$F("municipio")`
`$$('selector')`	Selección con **selectores CSS**	`$$('div.alerta')`
`$A(coso)`	Convierte en array algo "parecido a un array"	`$A(document.forms)`
`$H(objeto)`	Crea un array asociativo (hash)	`$H({a: 1, b: 2})`
`$R(desde, hasta)`	Crea un rango de valores	`$R(1, 10)`

`$F()`: valor de un campo

```
// Con JavaScript
document.getElementById("municipio").value

// Con Prototype
$F("municipio")
```

`$$()`: selectores CSS

```
$$('p')                // todos los párrafos
$$('#menu a')          // los enlaces dentro de #menu
$$('input[type=text]') // inputs de texto
```

`$A()`, `$H()`, `$R()`

```
var lista = $A(document.getElementsByTagName('li'));
lista.each(function (item) { console.log(item.innerHTML); });

var hash = $H({ nombre: "Ana", edad: 25 });
hash.each(function (par) { console.log(par.key + " = " + par.value); });

$R(1, 5).each(function (n) { console.log(n); });  // 1 2 3 4 5
```

> Documentación oficial: http://prototypejs.org

5. Prototype: AJAX (`Ajax.Request`, `Ajax.Updater`)

`Ajax.Updater`

Hace la petición y **vuelca la respuesta dentro** de un elemento (el contenedor que se le indica).

```
new Ajax.Updater('comprobar_mensaje', url, {
  method: 'get',
  parameters: "nickname=" + nick
});
```

Parámetro	Significado
`'comprobar_mensaje'`	`id` del elemento donde se inserta la respuesta.

Parámetro	Significado
`url`	Dirección del script del servidor.
`method`	`'get'` o `'post'`.
`parameters`	Datos a enviar (`"clave=valor"` o un objeto).

`Ajax.Request`

Igual, pero la respuesta se maneja en un **callback**, no se vuelca automáticamente.

```
new Ajax.Request(url, {
  method: 'post',
  parameters: { nickname: nick },
  onSuccess: function (transporte) {
    $('comprobar_mensaje').innerHTML = transporte.responseText;
  },
  onFailure: function () {
    $('comprobar_mensaje').innerHTML = "Error";
  }
});
```

Ejemplo completo (validar nick)

```
<script src="prototype.js" type="text/javascript"></script>
<script type="text/javascript">
function comprobar(nick) {
  var url = 'http://' + location.host + '/ajax/ajax_comprobar_nick.php';
  var pars = ("nickname=" + nick);

  var myAjax = new Ajax.Updater('comprobar_mensaje', url, {
    method: 'get',
    parameters: pars
  });
}
</script>

<input name="nickname" onkeyup="comprobar(this.value)">
<span id="comprobar_mensaje"></span>
```

Callback	Cuándo se ejecuta
`onCreate`	Al crear la petición.
`onSuccess`	Respuesta correcta.
`onFailure`	Error en la petición.
`onComplete`	Siempre, al terminar (éxito o error).
`onException`	Ante una excepción de JavaScript.

---

6. jQuery: qué es y para qué sirve

jQuery es una **biblioteca multiplataforma de JavaScript**, fácil de aprender.

Su propósito es **facilitar el uso de JavaScript** en el sitio web: simplifica tareas complicadas como las llamadas **AJAX** y la **manipulación del DOM**.

Características

Característica	Para qué
Manipulación **HTML / DOM**	Seleccionar, crear, mover y eliminar elementos.
Manipulación **CSS**	Clases y propiedades de estilo.
Métodos de eventos HTML	Clic, teclado, formularios, ventana.
Efectos y animaciones	Mostrar, ocultar, desvanecer, deslizar, animar.
AJAX	`load` , `get` , `post` , `ajax` .
Utilidades	Iteración, datos, *timers*, etc.

¿Por qué jQuery?

- Es probablemente la librería **más popular** y la **más ampliable**.

- La usan empresas grandes: Google, Microsoft, IBM, Netflix.
- La usan gestores de contenido como **WordPress** y **Joomla**.
- Es la base de frameworks de interfaz como **Bootstrap** y **Material Design**.

---

7. Formas de incluir jQuery

1. Descargando la librería

```
<head>
  <script src="jquery-3.5.1.min.js"></script>
</head>
```

2. Desde un CDN (Content Delivery Network)

```
<script src="https://ajax.googleapis.com/ajax/libs/jquery/3.5.1/jquery.min.js"></script>
```

3. Con un gestor de paquetes

```
npm install jquery
# o
yarn add jquery
```

Forma	Ventaja	Desventaja
Local	Funciona sin Internet; control de versión.	Hay que descargarla y actualizarla a mano.
CDN	Carga rápida y cacheada; cero configuración.	Requiere conexión.
npm / yarn	Integrada al build del proyecto.	Necesita tooling (bundler).

> Sitios oficiales: https://jquery.com — https://jquery.com/download

---

8. Sintaxis: `$(selector).action()`

```
$(selector).action()
```

Parte	Significado
`$`	Define / accede a jQuery.
`(selector)`	"Consulta" o busca elementos HTML.
`.action()`	Acción jQuery que se realiza sobre esos elementos.

```
$(this).hide()      // oculta el elemento actual
$("p").hide()       // oculta todos los <p>
$(".test").hide()   // oculta todos los elementos con class="test"
$("#test").hide()   // oculta el elemento con id="test"
```

---

9. `$(document).ready()`

Todos los métodos deben cargarse dentro de la función **ready** para evitar que los eventos se ejecuten antes de que termine la carga de la página.

```
$(document).ready(function () {
  // colocar aquí los métodos jQuery
});
```

Forma abreviada:

```
$(function () {
  // colocar aquí los métodos jQuery
});
```

> Es el equivalente a `window.onload`, pero se dispara cuando el DOM está listo, sin esperar imágenes ni recursos.

---

10. Selectores

```
$("button").click(function () {
  $("p").hide();
});
```

Selector	Selecciona
`$("*")`	Todos los elementos.
`$(this)`	El elemento actual.
`$("p")`	Todos los `<p>`.
`$("p.intro")`	Los `<p>` con `class="intro"`.
`$("p:first")`	El primer `<p>`.
`$("tr:odd")`	Las filas impares de una tabla.
`$("#id")`	El elemento con ese `id`.
`$(".clase")`	Los elementos con esa clase.
`$("ul li")`	Los `<li>` dentro de un `<ul>`.
`$("input[type=text]")`	Por atributo.

```
$(document).ready(function () {
  $("button").click(function () {
    $("p").hide();
  });
});
```

11. Eventos

Todas las acciones que el usuario realiza sobre la página pueden capturarse con eventos:

- Eventos del **mouse**.
- Eventos del **teclado**.
- Eventos de **formularios**.
- Eventos de la **ventana del navegador**.

Sintaxis

```
$("p").click();          // captura el evento
```

El segundo paso es definir **qué debe suceder** cuando se dispara: se pasa una función.

```
$("p").click(function () {
  // acciones
  $(this).hide();
});
```

Eventos frecuentes

Categoría	Eventos
Mouse	`click`, `dblclick`, `mouseenter`, `mouseleave`, `hover`, `mousedown`, `mouseup`
Teclado	`keypress`, `keydown`, `keyup`
Formulario	`submit`, `change`, `focus`, `blur`, `select`
Documento / ventana	`ready`, `load`, `resize`, `scroll`, `unload`

```
$("#nick").keyup(function () {
  var pnickname = $("#nick").val();
  console.log(pnickname);
});

$("#formulario").submit(function (e) {
  e.preventDefault();     // evita el envío tradicional
  // validar y procesar
});
```

12. Efectos y animaciones

`hide()` y `show()`

```
$("#hide").click(function () {
  $("p").hide();
});

$("#show").click(function () {
  $("p").show();
});
```

```
$(selector).hide(speed, callback);
$(selector).show(speed, callback);
$(selector).toggle(speed, callback);
```

Fade (desvanecido)

Método	Efecto
`fadeIn()`	Aparece gradualmente.
`fadeOut()`	Desaparece gradualmente.
`fadeToggle()`	Alterna entre uno y otro.
`fadeTo(speed, opacity)`	Lleva a una opacidad determinada.

```
$("#caja").fadeIn(1000);
$("#caja").fadeOut("slow");
$("#caja").fadeToggle(400);
$("#caja").fadeTo(500, 0.5);
```

Slide (deslizamiento)

Método	Efecto
`slideDown()`	Se despliega hacia abajo.
`slideUp()`	Se repliega hacia arriba.
`slideToggle()`	Combina los dos anteriores.

```
$("#panel").slideToggle();
```

`animate()`

```
$(selector).animate({params}, speed, callback);
```

```
$("#caja").animate({
  left: "250px",
  opacity: 0.5,
  height: "150px"
}, 1000);
```

`stop()`

Detiene las animaciones o efectos en curso.

```
$("#caja").stop();
```

Velocidades

Valor	Milisegundos
`"slow"`	600
`"fast"`	200
número	Los ms indicados

13. Encadenamiento y callbacks

Callbacks

Permiten que el script se ejecute **paso a paso**: la función se llama cuando el efecto termina.

```
$("p").hide(1000, function () {
  alert("El párrafo ya está oculto");
});
```

Encadenamiento (*chaining*)

Con jQuery se pueden encadenar acciones sobre el mismo elemento:

```
$("#caja")
  .css("color", "red")
  .slideUp(500)
  .slideDown(500);
```

> Cada método devuelve el objeto jQuery, por eso se puede seguir llamando métodos.

---

14. Manipulación de HTML: obtener y setear

jQuery contiene métodos para cambiar y manipular elementos y atributos HTML.

Obtener (get)

Método	Devuelve
`text()`	El contenido **de texto** de los elementos seleccionados.
`html()`	El contenido **incluido el marcado HTML**.
`val()`	El **valor** de los campos de formulario.
`attr("nombre")`	El valor de un atributo.

```
$("button").click(function () {
  alert($("#w3s").attr("href"));
});
```

Setear (set)

Se usan **los mismos métodos**, pasando el nuevo valor.

```
$(document).ready(function () {
  $("#btn1").click(function () {
    $("#test1").text("Hola mundo!");
  });

  $("#btn2").click(function () {
    $("#test2").html("<b>Hello world!</b>");
  });

  $("#btn3").click(function () {
    $("#test3").val("Dolly Duck");
  });
});
```

Método	Get	Set
`text()`	`$("p").text()`	`$("p").text("nuevo")`
`html()`	`$("p").html()`	`$("p").html("<b>nuevo</b>")`
`val()`	`$("input").val()`	`$("input").val("nuevo")`
`attr()`	`$("a").attr("href")`	`$("a").attr("href", "url")`

> `text()` escapa el HTML; `html()` lo interpreta. Nunca uses `html()` con datos del usuario sin sanitizar (XSS).

---

15. Agregar contenido

Método	Dónde inserta
`append()`	Al **final**, dentro de los elementos seleccionados.
`prepend()`	Al **comienzo**, dentro de los elementos seleccionados.
`after()`	**Después** de los elementos seleccionados.
`before()`	**Antes** de los elementos seleccionados.

```
$("p").append("algún texto.");
$("p").prepend("algún texto.");
$("img").after("Some text after");
$("img").before("Some text before");
```

```
before()  →  [ elemento ]  →  after()
             prepend() ... append()
```

16. Eliminar contenido

Método	Qué elimina
`remove()`	El elemento seleccionado **y sus hijos**.
`empty()`	Solo los **elementos secundarios**, deja el contenedor.

```
$("#div1").remove();
$("#div1").empty();

$("p").remove(".test");   // remueve los <p> con class="test"
```

17. Manipulación de CSS

Método	Función
`addClass()`	Agrega una o más clases.
`removeClass()`	Elimina una o más clases.
`toggleClass()`	Alterna agregar / eliminar.
`css()`	Establece o devuelve propiedades de estilo.

```
$("p").addClass("destacado");
$("p").removeClass("destacado");
$("p").toggleClass("destacado");
```

`css()`

```
// Retornar un valor
$("p").css("background-color");

// Establecer un valor
$("p").css("background-color", "yellow");

// Establecer varias propiedades
$("p").css({
  "background-color": "yellow",
  "font-size": "200%"
});
```

18. AJAX con jQuery: `load()`

Es el método **más simple** de AJAX con jQuery. El resultado se coloca en un contenedor.

```
$(selector).load(URL, data, callback);
```

Parámetro	Descripción
`URL`	**Requerido.** Dirección que se desea llamar.

Parámetro	Descripción
`data`	Opcional. Duplas clave/valor que se envían al servidor.
`callback`	Opcional. Función que se ejecuta al completarse `load()`.

```
<script src="https://ajax.googleapis.com/ajax/libs/jquery/3.5.1/jquery.min.js"></script>
<script>
$(document).ready(function () {
  $("button").click(function () {
    $("#div1").load("Holamundo.txt");
  });
});
</script>
```

> En `load()` la respuesta se inserta como HTML. (La teoría aclara que si solo se quiere mostrar texto sin interpretar etiquetas, conviene otro método.)

Validar nickname con `load()`

```
$(document).ready(function () {
  $("#nick").keyup(function () {
    var pnickname = $("#nick").val();

    $("#comprobar_mensaje").load(
      "ajax_comprobar_nick.php",
      { nickname: pnickname },
      function (response, status, xhr) {
        if (status == "error") {
          var msg = "Error!, algo ha sucedido: ";
        }
      }
    );
  });
});
```

---

19. AJAX con jQuery: `get()` y `post()`

Se usan para solicitar datos con una petición HTTP **GET** o **POST**.

	GET	POST
Propósito	**Obtener / recuperar** datos.	**Enviar** datos para su procesamiento.
Caché	Puede devolver datos cacheados.	**Nunca** almacena en caché.
Datos	En la URL.	En el cuerpo.

> Ambos **respetan el contenido devuelto**: si la respuesta tiene etiquetas HTML, el navegador las interpreta.

`$.get()`

```
$.get(URL, callback);
```

```
$("button").click(function () {
  $.get("holamundo.txt", function (data, status) {
    alert("Data: " + data + "\nStatus: " + status);
  });
});
```

```
$("#nick").keyup(function () {
  var pnickname = $("#nick").val();

  $.get("ajax_comprobar_nick.php?nickname=" + pnickname,
    function (data, status) {
      $("#comprobar_mensaje").html(data);
    });
});
```

`$.post()`

```
$.post(URL, data, callback);
```

```
$("button").click(function () {
  $.post("post.php",
    {
      nombre: "Jose Lopez",
      ciudad: "General Pico"
    },
    function (data, status) {
      alert("Data: " + data + "\nStatus: " + status);
    });
});
```

```
$("#nick").keyup(function () {
  var pnickname = $("#nick").val();

  $.post("ajax_comprobar_nick.php",
    { nickname: pnickname },
    function (data, status) {
      $("#comprobar_mensaje").html(data);
    });
});
```

Resumen

Método	Sintaxis	Cuándo usarlo
`load()`	`$(sel).load(url, data, cb)`	Insertar HTML directamente en un contenedor.
`get()`	`$.get(url, cb)`	Traer datos sin modificar el servidor.
`post()`	`$.post(url, data, cb)`	Enviar datos (altas, login, actualizaciones).
`ajax()`	`$.ajax({...})`	Control total: cabeceras, timeouts, errores.

---

20. AJAX con jQuery: `ajax()`

Versión mínima

```
$(document).ready(function () {
  $("button").click(function () {
    $.ajax({
      url: "holamundo.txt",
      success: function (result) {
        $("#div1").html(result);
      }
    });
  });
});
```

Versión completa

```
$.ajax({
  url: "ruta/del/servidor.php",       // URL de la petición
  type: "GET",                        // GET, POST, PUT, DELETE
  data: { nombre: "Juan", edad: 25 }, // datos enviados
  dataType: "json",                   // json, text, html, xml
  timeout: 5000,                      // espera máxima (ms)

  beforeSend: function (xhr) {
    console.log("Enviando petición...");
  },

  success: function (response, status, xhr) {
    console.log("Respuesta:", response);
    console.log("Estado:", status);       // "success"
    console.log("Código HTTP:", xhr.status);
  },

  error: function (xhr, status, error) {
    console.error("Error:", error);
    console.error("Estado:", status);
    console.error("Código HTTP:", xhr.status);
  },

  complete: function (xhr, status) {
    console.log("Petición finalizada con estado:", status);
```

```
  }
});
```

Parámetros más comunes

Parámetro	Función
`url`	Dirección a la que se envía la petición.
`type` / `method`	Método HTTP (`GET` , `POST` , etc.).
`data`	Datos a enviar al servidor.
`dataType`	Formato esperado de la respuesta (`json` , `xml` , `html` , `text`).
`timeout`	Tiempo máximo de espera.
`beforeSend`	Callback antes de enviar.
`success`	Callback cuando la petición se resuelve correctamente.
`error`	Callback cuando ocurre un error.
`complete`	Callback al final, con éxito o error.

Con `done()` / `fail()`

```
$.ajax({
  type: 'POST',
  url: './ajax_comprobar_nick.php',
  data: { nickname: pnickname }
})
.done(function (msg) {
  $("#comprobar_mensaje").html(msg);
})
.fail(function (jqXHR, textStatus, errorThrown) {
  $("#comprobar_mensaje").html(
    "Ocurrió un error: " + textStatus + " " + errorThrown
  );
});
```

> `done()` y `fail()` trabajan con **promesas**; `success` y `error` son las opciones clásicas. No hace falta usar ambas.

---

21. `noConflict()`

El problema

jQuery usa `$` como atajo. Muchos otros frameworks también lo usan (**Angular, Backbone, Ember, Knockout, Prototype**). Si dos frameworks usan el mismo atajo, uno puede dejar de funcionar.

La solución

`noConflict()` **libera** el uso de `$` para que otros scripts puedan utilizarlo. Después se usa el nombre completo `jQuery`.

```
$.noConflict();

jQuery(document).ready(function () {
  jQuery("button").click(function () {
    jQuery("p").text("jQuery is still working!");
  });
});
```

Crear un atajo propio

`noConflict()` devuelve una referencia a jQuery:

```
var jq = $.noConflict();

jq(document).ready(function () {
  jq("button").click(function () {
    jq("p").text("jQuery is still working!");
  });
});
```

Situación	Solución
Solo se usa jQuery	Dejar `$` como atajo.
Convive con Prototype u otro	`$.noConflict()` y usar `jQuery(...)`.
Se quiere un atajo corto propio	`var jq = $.noConflict();`

22. Prototype vs. jQuery

Aspecto	Prototype	jQuery
Selección por `id`	`$('id')`	`$('#id')`
Selección CSS	`$$('sel')`	`$('sel')`
Valor de un campo	`$F('id')`	`$('#id').val()`
Evento	`Event.observe(...)`	`.click()`, `.on()`
Efectos	`Element.hide()`, `Effect.*`	`.hide()`, `.fadeIn()`, `.animate()`
AJAX	`Ajax.Request`, `Ajax.Updater`	`$.ajax`, `$.get`, `$.post`, `.load()`
Estilo	Extiende objetos del DOM	Envuelve elementos en un objeto jQuery
Popularidad actual	Baja (heredada)	Alta, aunque desplazada por frameworks modernos

> Idea para el parcial: **Prototype** es la base histórica de AJAX del lado del cliente; **jQuery** generalizó el enfoque con `$(selector).accion()` y una API de AJAX muy compacta.

23. Errores típicos y checklist

Error	Causa	Solución
`$ is not a function`	jQuery no se cargó o se cargó después del script.	Incluir jQuery en el `<head>` **antes** del código propio.
Nada funciona y no hay errores	El código corre antes de que exista el DOM.	Envolver en `$(document).ready()` o `$(function(){})`.
El selector no encuentra nada	`id`/clase mal escrita o elemento inexistente.	Verificar en DevTools con `$("#id").length`.
`$` choca con otro framework	Prototype u otro también usa `$`.	`$.noConflict()` y usar `jQuery`.
El evento se dispara muchas veces	Se registró el handler varias veces (p. ej. dentro de otro evento).	Registrar una sola vez o usar `.off()` antes.
`val()` devuelve `undefined`	El selector no apunta a un campo de formulario.	Revisar el selector.
Se inyecta HTML del usuario	Se usó `.html()` con datos sin sanitizar.	Usar `.text()` o sanitizar (XSS).
La animación se encola	Se hicieron clics repetidos.	`.stop()` antes de animar.
AJAX devuelve 404	Ruta relativa mal armada.	Revisar la URL desde la página actual.
`$.post` no llega al PHP	Se esperaba `$_GET` o el `name` no coincide.	Revisar método y nombres de campos.
`load()` inserta todo el HTML	Es su comportamiento.	Usar `$.get` + `.text()` si solo se quiere texto.

Checklist antes de entregar

- [ ] jQuery incluida **una sola vez** y antes del código propio.
- [ ] Todo el código dentro de `$(document).ready()`.
- [ ] Selectores probados (existen y son únicos los `id`).
- [ ] Eventos con `preventDefault()` cuando corresponda.
- [ ] Efectos con `speed` coherente y `stop()` si se repiten.
- [ ] `.text()` para datos del usuario; `.html()` solo con contenido confiable.
- [ ] Peticiones AJAX con manejo de error (`.fail()` o `error`).
- [ ] `noConflict()` si convive con Prototype u otro framework.
- [ ] Sin `console.log` de depuración en la entrega.
- [ ] Funciona con la consola del navegador limpia (sin errores).

24. Plantilla completa

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>jQuery — Plantilla</title>

  <!-- 1. jQuery desde CDN -->
  <script src="https://ajax.googleapis.com/ajax/libs/jquery/3.5.1/jquery.min.js"></script>

  <script>
  // 2. Todo dentro de ready
  $(function () {

    // 3. Evento
    $("#btnCargar").click(function () {
      $("#salida").load("contenido.html");
    });

    $("#btnOcultar").click(function () {
      $("p").stop().slideToggle(300);
    });

    $("#nick").keyup(function () {
      var nick = $(this).val();

      if (nick.length === 0) {
        $("#mensaje").text("");
        return;
      }

      // 4. AJAX con control de errores
      $.ajax({
        type: "POST",
        url: "ajax_comprobar_nick.php",
        data: { nickname: nick },
        dataType: "text"
      })
      .done(function (respuesta) {
        $("#mensaje").text(respuesta);
      })
      .fail(function (xhr, status, error) {
        $("#mensaje").text("Error: " + status + " " + error);
      });
    });

    // 5. Estilos y clases
    $("#btnDestacar").click(function () {
      $("article").toggleClass("destacado");
      $("article").css({ "border-left": "4px solid #b45309" });
    });

  });
  </script>
</head>
<body>

  <button id="btnCargar">Cargar contenido</button>
  <button id="btnOcultar">Mostrar / ocultar</button>
  <button id="btnDestacar">Destacar</button>

  <input type="text" id="nick" placeholder="Nickname">
  <span id="mensaje"></span>

  <div id="salida"></div>

  <article>
    <p>Texto de ejemplo para los efectos.</p>
  </article>

</body>
</html>
```

Equivalente mínimo en Prototype

```
<script src="prototype.js" type="text/javascript"></script>
<script type="text/javascript">
function comprobar(nick) {
  var url = 'http://' + location.host + '/ajax/ajax_comprobar_nick.php';
  new Ajax.Updater('comprobar_mensaje', url, {
    method: 'get',
    parameters: "nickname=" + nick
  });
}
</script>

<input name="nickname" onkeyup="comprobar(this.value)">
<span id="comprobar_mensaje"></span>
```

Fin del cheatsheet.

Si tenés que priorizar para el parcial: funciones `$`, `$F`, `$$`, `$A`, `$H`, `$R` de Prototype y `Ajax.Updater`; en jQuery la sintaxis `$(selector).action()`, `ready`, selectores, eventos, `hide/show` + fade + slide + `animate`, `text/html/val/attr`, `append/prepend/after/before`, `addClass/removeClass/toggleClass/css`, los cuatro métodos AJAX (`load`, `get`, `post`, `ajax`) y `noConflict()`.